



PRÉ-RENTRÉE 2026-2027

SÉANCE 3 - CONTRATS, TESTS ET FIABILITÉ DU CODE

Dossier personnalisé - Ahmed BENHADJ SALEM - durée : 2 heures

Parcours : Fiabilisation et autonomie - candidat individuel

Objectif personnel de la séance 3 : transformer une spécification en fonctions courtes, expliciter leurs contrats, écrire les tests **avant** le code et couvrir les cas normaux, limites et invalides.

Production à conserver : le fichier `S3_fonctions_tests_Ahmed.py`, contenant quatre fonctions documentées, au moins **dix assertions**, un choix explicite pour la liste vide et une vérification de flottant avec `math.isclose`.

Contrat de fiabilisation

1. Je rédige l'interface, la précondition et la postcondition avant l'implémentation.
2. Je fixe les résultats attendus avant de lancer le programme.
3. Je couvre au minimum : cas normal, égalité, borne, entrée vide ou invalide.
4. Je refuse les dépendances inutiles à une variable globale.
5. Je ne confonds pas exactitude mathématique et représentation approchée des flottants.

1. Feuille de route des 120 minutes

Temps	Travail	Preuve attendue
0–10 min	Rituel chronométré : retour, interface et tests.	Prévisions exactes sans exécution.
10–30 min	Portée locale, état global et audit d'interface.	Bug reproduit et architecture corrigée.
30–48 min	<code>return</code> , <code>None</code> et décisions de contrat.	Comportement vide choisi et justifié.
48–65 min	Préconditions, postconditions et matrice de tests.	Dix résultats attendus fixés.
65–70 min	Pause.	–
70–100 min	Module de quatre fonctions.	Code documenté et exécutable.
100–112 min	Flottants, couverture et débogage.	<code>math.isclose</code> et cas limites validés.
112–120 min	Revue de code et oral bref.	Deux fonctions expliquées sans lecture mot à mot.

Critères de réussite : contrats cohérents ; dix assertions conservées ; cas vide traité conformément au choix annoncé ; aucune variable globale modifiée par une fonction de calcul ; test de flottant robuste ; correction minimale documentée.

2. Rituel de fiabilité - 10 min

Contraintes : 6 minutes de prévision sans ordinateur, puis 4 minutes d'exécution et de comparaison. Ne pas modifier une réponse attendue après avoir vu le résultat.

Rituel 1 - sortie affichée et valeur renvoyée

```
def f(x):
    print(x + 1)

r = f(4)
```

Question	Réponse avant exécution
Affichage produit	
Valeur de r	
Type de r	
Modification minimale pour renvoyer 5	

Rituel 2 - interface complète

```
def distance(a: float, b: float) -> float:
    """Renvoie la distance entre a et b."""
    return abs(a - b)

resultat = distance(-2.5, 4.0)
```

Compléter :

- paramètres : _____ ; arguments : _____ ;
- précondition utile : _____ ;
- postcondition : _____.

Rituel 3 - tests avant code

Pour `maximum_deux(a, b)`, écrire trois appels qui couvrent exactement :

1. deux valeurs distinctes positives : _____
2. deux valeurs égales : _____
3. deux valeurs négatives : _____

Certitude : ☐1 ☐2 ☐3 ☐4

Contrôle : _____

3. Portée locale et suppression de l'état global - 20 min

Règle d'architecture. Une fonction de calcul doit recevoir ses données par ses paramètres et transmettre son résultat par `return`. Une variable globale modifiable rend les tests dépendants de l'ordre d'exécution.

A. Reproduire le bug

```
compteur = 0

def compte_superieurs(valeurs, seuil):
    for valeur in valeurs:
        if valeur >= seuil:
            compteur += 1
    return compteur

print(compte_superieurs([2, 5, 8], 5))
```

1. Prévoir : résultat numérique ou exception ? _____
2. Nom exact de l'état qui ne devrait pas être global : _____
3. Écrire le plus petit jeu de test qui reproduit le problème.

B. Corriger l'architecture, pas seulement le symptôme

Réécrire uniquement les lignes nécessaires pour que :

- le compteur soit initialisé à chaque appel ;
- aucun mot-clé `global` ne soit utilisé ;
- deux appels successifs avec les mêmes arguments donnent le même résultat.

C. Test d'indépendance des appels

Prévoir puis exécuter :

```
a = compte_superieurs([2, 5, 8], 5)
b = compte_superieurs([2, 5, 8], 5)
assert a == b
```

Pourquoi ce test serait-il fragile avec un compteur global conservé entre les appels ?

Aides graduées - une seule à la fois : A - nommer les entrées et la sortie ; B - distinguer `print` et `return` ; C - repérer les variables locales ; D - écrire la précondition et la postcondition ; E - ajouter un cas normal, un cas limite et un cas invalide.

4. Choisir et annoncer le contrat de moyenne - 18 min

Décision obligatoire. Une moyenne sur liste vide n'est pas définie. Le programme doit donc annoncer un comportement précis et le respecter dans le code comme dans les tests.

A. Comparer deux contrats recevables

Choix	Comportement	Conséquence pour l'appelant
A - précondition	<code>assert len(valeurs) > 0</code>	l'entrée vide est refusée explicitement
B - valeur spéciale	<code>return None</code> si la liste est vide	l'appelant doit tester le résultat avant tout calcul

Contrat retenu pour tout le fichier : ☐A ☐B

Justification en termes d'usage :

B. Écrire l'interface complète

Rubrique	Rédaction d'Ahmed
Signature proposée	
Type et rôle du paramètre	
Précondition	
Postcondition	
Comportement sur liste vide	

C. Prévoir les cas avant le code

Catégorie	Entrée	Résultat ou comportement attendu
Cas normal	[10, 14, 18]	
Singleton	[7]	
Valeurs négatives	[-4, -2, -6]	
Liste vide	[]	
Flottants	[0.1, 0.2, 0.3]	

5. Matrice de tests avant implémentation - 17 min

Consigne : remplir toutes les colonnes « attendu » avant d'ouvrir le starter. Les valeurs attendues deviennent ensuite des assertions ; elles ne sont pas adaptées au code obtenu.

N°	Fonction	Entrée	Attendu
1	est_pair	8	
2	est_pair	-3	
3	maximum_deux	5, 5	
4	maximum_deux	-4, -9	
5	moyenne	[10, 14, 18]	
6	moyenne	[7]	
7	moyenne	[]	
8	compte_superieurs	[2,5,8], 5	
9	compte_superieurs	[], 5	
10	compte_superieurs	[5], 5	

Couverture attendue

Associer chaque numéro à une catégorie : normal, négatif, égalité, vide, borne exacte.

Catégorie	Numéro(s)
Cas normal	
Valeur négative	
Égalité	
Liste vide	
Valeur exactement égale au seuil	

Assertion et exception

Pour le contrat A, compléter le premier gabarit ; pour le contrat B, compléter le second. Ne conserver dans le fichier que la version correspondant au choix de la page 4.

```
# Contrat A : l'appel vide doit provoquer AssertionError.
try:
    moyenne([])
    assert False, "La liste vide aurait du etre refusee"
except AssertionError:
    pass

# Contrat B : l'appel vide renvoie une valeur speciale.
assert moyenne([]) is _____
```

Avant la pause : dix attentes sont fixées et chaque cas limite a une raison d'être. Une fonction non encore écrite peut déjà être testée sur le papier.

6. Module principal : trois fonctions fiables - 20 min

Ouvrir : S3_fonctions_tests_Ahmed.py. Les tests doivent être écrits avant le corps définitif de chaque fonction, puis réactivés à mesure que le code est complété.

A. est_pair - échauffement de 3 minutes

Contraintes : une docstring, une seule instruction `return`, aucun `if` nécessaire.

```
def est_pair(n: int) -> bool:
    """Renvoie True si n est pair, False sinon."""
    pass
```

B. maximum_deux - égalité et négatifs

Contraintes : tous les chemins renvoient ; aucune valeur sentinelle comme 0.

```
def maximum_deux(a, b):
    """Renvoie le plus grand des deux nombres."""
    pass
```

C. moyenne - contrat annoncé

Contraintes : calculer la somme par une boucle afin de contrôler l'accumulateur ; un seul parcours ; comportement vide conforme à la page 4.

```
def moyenne(valeurs):
    """Contrat a completer avant le code."""
    pass
```

D. Revue immédiate

Fonction	Docstring	Return	Tests verts
est_pair	☐	☐	☐
maximum_deux	☐	☐	☐
moyenne	☐	☐	☐

Interdiction utile : ne pas utiliser `sum`, `max` ni une variable globale pour masquer une difficulté déjà travaillée lors de S2.

7. Quatrième fonction : seuil et cas limite - 10 min

Spécification de `compte_superieurs(valeurs, seuil)` : renvoyer le nombre de valeurs **supérieures ou égales** au seuil. Une liste vide renvoie 0. Une valeur exactement égale au seuil doit être comptée.

A. Tests obligatoires avant code

```
assert compte_superieurs([2, 5, 8], 5) == _____
assert compte_superieurs([], 5) == _____
assert compte_superieurs([5], 5) == _____
assert compte_superieurs([-4, -1, 0], -1) == _____
```

B. Pseudo-code précis

1. État initial : _____
2. Invariant après k tours : _____

3. Mise à jour : _____
4. Valeur renvoyée : _____

C. Implémentation sans état global

D. Audit de la borne

Remplacer temporairement `>=` par `>`, lancer seulement le test `[5], 5`, consigner le symptôme, puis restaurer la spécification correcte.

Symptôme	Cause	Correction minimale
----------	-------	---------------------

8. Défi chronométré : deuxième maximum distinct - 12 min

Spécification. `deuxieme_plus_grand_distinct(valeurs)` renvoie la deuxième plus grande valeur **distincte**, ou `None` s'il existe moins de deux valeurs distinctes. Les appels ne doivent pas modifier la liste reçue.

Contraintes : 12 minutes; tests avant code; pas de `sort`, pas de `sorted`, pas de conversion en ensemble; une architecture explicable.

A. Jeux de tests fixés

N°	Entrée	Attendu	Catégorie
1	[8, 3, 8, 5]		répétition du maximum
2	[4]		une seule valeur
3	[7, 7]		aucune seconde valeur distincte
4	[-2, -5, -3]		valeurs négatives
5	[]		liste vide

B. État à maintenir pendant le parcours

Variable	Signification après le préfixe déjà parcouru
<code>plus_grand</code>	
<code>deuxieme</code>	

C. Plan puis code

Écrire d'abord les cas de mise à jour en français, notamment lorsque la valeur est égale au maximum actuel.

Implémentation dans le fichier : ☐terminée ☐partielle mais tests conservés.

D. Explication de complexité

Nombre de parcours de la liste : _____. Coût attendu en fonction de `n` : _____. Justification :

9. Flottants, couverture et journal de débogage - 12 min

A. Égalité approchée

Exécuter :

```
print(0.1 + 0.2 == 0.3)
```

Puis utiliser :

```
from math import isclose
assert isclose(0.1 + 0.2, 0.3)
```

1. Résultat du test direct : _____
2. Pourquoi `isclose` est-il mieux adapté ici ?

3. Écrire un test robuste pour la moyenne de `[0.1, 0.2, 0.3]`.

B. Carte de couverture du module

Fonction	normal	égalité	négatif	vide	borne
<code>est_pair</code>	☐	☐	☐	–	–
<code>maximum_deux</code>	☐	☐	☐	–	–
<code>moyenne</code>	☐	–	☐	☐	☐
<code>compte_superieurs</code>	☐	–	☐	☐	☐
<code>deuxieme_plus_grand</code>	☐	☐	☐	☐	–
<code>distinct</code>					

C. Journal de débogage

Test rouge	Cause supposée	Modification unique	Nouveau résultat
------------	----------------	---------------------	------------------

Aides graduées - une seule à la fois : A - nommer les entrées et la sortie ; B - distinguer `print` et `return` ; C - repérer les variables locales ; D - écrire la précondition et la postcondition ; E - ajouter un cas normal, un cas limite et un cas invalide.

10. Revue de code, oral et exit ticket - 8 min

A. Revue finale du fichier

Critère	Validé
Chaque fonction possède une interface et une docstring cohérentes.	☐
Les préconditions et postconditions sont compatibles avec les tests.	☐
La décision sur la liste vide est identique dans le texte, le code et les assertions.	☐
Aucune fonction de calcul ne dépend d'une variable globale modifiée.	☐
Au moins dix assertions couvrent plusieurs catégories de cas.	☐
Les flottants sont testés avec <code>isclose</code> lorsque nécessaire.	☐
Le module s'exécute de haut en bas sans intervention manuelle.	☐
Une copie datée est conservée.	☐

B. Oral de 2 minutes

Choisir deux fonctions et préparer, pour chacune :

1. les entrées et la sortie ;
2. la décision de contrat ;
3. le cas limite le plus révélateur ;
4. un bug rencontré et la preuve de sa correction.

Fonctions choisies : _____ et _____

C. Exit ticket - sans exécution

1. Une fonction sans `return` exécuté renvoie : _____
2. Donner une précondition recevable pour une fonction qui renvoie le minimum d'une liste.

3. Pourquoi un jeu de tests réussi ne prouve-t-il pas l'absence de tous les bugs ?

4. Écrire un test de borne pour `compte_supérieurs`.

5. Donner une raison d'éviter une variable globale modifiable dans une fonction de calcul.

Contrôle que je rends systématique dès septembre :

Preuve d'autonomie conservée aujourd'hui :
